Code Structure & Custom Plugin — Technical Reference

This document explains **every folder, file, and module** in the repository and how they connect together.

Repository Structure at a Glance

```
peptive/
├── app/                          # Next.js App Router (pages & API routes)
│   ├── layout.tsx                # Root layout (header, footer, providers)
│   ├── page.tsx                  # Homepage
│   ├── not-found.tsx             # Custom 404 page
│   ├── globals.css               # Global styles + Tailwind directives
│   ├── account/                  # User account dashboard
│   ├── api/                      # Backend API routes (server-side)
│   │   ├── calculate-shipping-tax/   # Shipping cost calculation
│   │   ├── create-checkout-session/  # Stripe checkout session
│   │   ├── customer-addresses/       # Fetch customer addresses
│   │   ├── customer-orders/          # Fetch customer orders
│   │   ├── proxy/wc-store/           # CORS proxy for WooCommerce Store
API
│   │   ├── shipping-locations/       # Fetch shipping zones from
WooCommerce
│   │   ├── test-woocommerce/         # Debug endpoint to test WooCommerce
connection
│   │   ├── test-wordpress/           # Debug endpoint to test WordPress
connection
│   │   ├── validate-coupon/          # Validate WooCommerce coupons
│   │   ├── verify-code/              # Product authentication code
verification
│   │   └── webhooks/stripe/          # Stripe payment webhook handler
│   ├── cart/                     # Full cart page
│   ├── checkout/                 # Checkout page
│   │   ├── cancel/               # Payment cancelled page
│   │   └── success/              # Payment success page
│   ├── login/                    # User login page
```

```
│   ├── oral-peptides/          # Oral peptides category page
│   ├── pages/                  # Miscellaneous pages
│   │   ├── authentication/     # Product verification page
│   │   ├── dosage-calculator/  # Peptide dosage calculator
│   │   └── instant-authentication/  # Authentication success page
│   ├── privacy-policy/         # Privacy policy page
│   ├── products/               # Products listing
│   │   └── [slug]/             # Dynamic product detail page
│   ├── signup/                 # User registration page
│   └── stack/                  # Stack builder page
├── components/                 # Reusable React components
│   ├── analytics/              # Google Analytics & Meta Pixel
│   ├── cart/                   # CartItem, CartSidebar
│   ├── layout/                 # Header, Footer, AnnouncementBar
│   ├── products/               # ProductCard, ProductGrid,
RelatedProducts
│   ├── ui/                     # Button, Input (generic UI primitives)
│   ├── CountrySelector.tsx     # Country dropdown
│   ├── LanguageSelector.tsx    # Language dropdown (EN/AR)
│   ├── MobileLanguageToggle.tsx # Mobile EN/AR toggle
│   └── WelcomePopup.tsx        # Lead capture popup (→ Google Forms)
├── contexts/                   # React Context providers
│   └── LanguageContext.tsx      # i18n language state (EN/AR)
├── lib/                        # Core libraries and API clients
│   ├── auth.ts                 # JWT authentication client
│   ├── store-api.ts            # WooCommerce Store API client
│   ├── tracking.ts             # GA4 + Meta Pixel event tracking
│   ├── translations.ts         # English & Arabic translation strings
│   ├── useTracking.ts          # React hook for page view tracking
│   ├── utils.ts                # Utility functions (formatPrice,
validation, etc.)
│   ├── woocommerce.ts          # WooCommerce product/review API client
│   └── wordpress.ts            # WordPress CMS API client (pages, hero,
banners)
├── store/                      # State management
│   └── cartStore.ts            # Zustand cart store (persisted to
localStorage)
├── types/                      # TypeScript type definitions
│   └── index.ts                # All interfaces (Product, Cart,
StoreCart, etc.)
├── wordpress-plugin/           # Custom WordPress plugin (deploy to WP)
```

```
│   └── peptive-bundles/        # The plugin folder
├── UPDATED-FUNCTIONS.php       # WordPress functions.php additions for
CORS
├── next.config.mjs             # Next.js configuration
├── tailwind.config.ts          # Tailwind CSS configuration
├── tsconfig.json               # TypeScript configuration
└── package.json                # Dependencies and scripts
```

How the Code Flows (Request Lifecycle)

1. User Visits a Page

```
Browser → Next.js App Router → app/[page]/page.tsx → renders React
components
                                        │
                                        ├── Header
(components/layout/Header.tsx)
                                        ├── Page content (fetches data from
WooCommerce)
                                        ├── Footer
(components/layout/Footer.tsx)
                                        ├── CartSidebar
(components/cart/CartSidebar.tsx)
                                        └── WelcomePopup
(components/WelcomePopup.tsx)
```

2. User Browses Products

```
ProductsPage (app/products/page.tsx)
    │
    └── Calls woocommerce.getProducts() (lib/woocommerce.ts)
          │
          ├── Server-side: Direct call to WP Store API (/wp-
json/wc/store/v1/products)
          └── Client-side: Proxied through /api/proxy/wc-store/products
                │
                └── Proxy route (app/api/proxy/wc-
store/[...path]/route.ts)
                      forwards request to WordPress
```

3. User Adds to Cart

```
ProductDetailPage → addItem() → Zustand cartStore
                                        │
                                        ├── Updates state in memory
                                        ├── Persists to localStorage
                                        └── Opens CartSidebar overlay
```

The cart is **entirely client-side** using Zustand. No server calls needed. This makes it instant.

4. User Checks Out

```
CheckoutPage (app/checkout/page.tsx)
    │
    ├── Calculates shipping via POST /api/calculate-shipping-tax
    ├── Validates coupon via POST /api/validate-coupon
    │
    └── On submit: POST /api/create-checkout-session
          │
          ├── Creates Stripe Checkout Session with line items
          ├── Includes tax and shipping as separate line items
          └── Redirects user to Stripe hosted checkout page
```

5. Payment Completes

```
Stripe processes payment
    │
    ├── Redirects user to /checkout/success?session_id=xxx
    │     └── Clears cart, shows success message
    │
    └── Sends webhook POST to /api/webhooks/stripe
          │
          ├── Verifies Stripe signature
          ├── Extracts line items with product metadata
          └── Creates WooCommerce order via REST API
                (status: processing, set_paid: true)
```

Core Library Files Explained

`lib/woocommerce.ts` — Product Data

The main WooCommerce client. Uses **two API styles**:

API	Used For	Authentication
Store API (`/wc/store/v1/`)	Products (public), Cart	None (public)
REST API v3 (`/wc/v3/`)	Reviews, Coupons, Orders, Customers	Consumer Key/Secret

Key methods:

- `getProducts()` — Fetches product list from Store API
- `getProductBySlug()` — Fetches single product
- `getProductsByIds()` — Fetches related products
- `getFeaturedProducts()` — Featured products for homepage
- `getProductReviews()` — Reviews via REST API v3 (requires auth)

The `transformStoreProduct()` method converts WooCommerce's Store API format (prices in cents) to a clean frontend `Product` type. It also extracts custom plugin data from `extensions['peptive-bundles']`.

`lib/wordpress.ts` — CMS Content

Fetches WordPress pages, hero sections (via ACF), banners, and global settings. Used for CMS-managed content.

`lib/auth.ts` — User Authentication

JWT-based authentication client:

- `login()` — Gets JWT token from WordPress
- `register()` — Creates new user via custom REST endpoint
- `validateToken()` — Checks if stored token is still valid
- `getCurrentUser()` — Fetches user profile
- Token stored in `localStorage`

`lib/store-api.ts` — Cart & Checkout (Server-Side)

Direct WooCommerce Store API client for cart operations (add, update, remove items, apply coupons, checkout). This is a secondary client — the **primary cart** is Zustand (client-side).

`lib/translations.ts` — Bilingual Content

Contains all UI strings in English and Arabic. The `LanguageContext` reads from this file. Product content translations come from the WordPress plugin (stored as post meta).

`lib/tracking.ts` — Analytics

Centralized tracking for:

- **Google Analytics 4**: Page views, view_item, add_to_cart, begin_checkout, purchase
- **Meta Pixel**: ViewContent, AddToCart, InitiateCheckout, Purchase

All events fire in both platforms simultaneously.

`lib/utils.ts` — Utilities

Helper functions: `formatPrice()` (AED formatting), `stripHTML()`, validation, debounce, stock status labels.

`store/cartStore.ts` — Cart State

Zustand store with `persist` middleware:

- Items stored in `localStorage` under key `cart-storage`
- Each item gets a unique `cartItemId` (supports multiple bundle types of same product)
- Actions: `addItem`, `removeItem`, `updateQuantity`, `clearCart`, `toggleCart`
- Derived: `getItemCount()`, `getSubtotal()`

API Routes Explained

`/api/create-checkout-session` (POST)

Creates a Stripe Checkout Session. Transforms cart items into Stripe line items with proper product metadata (WooCommerce product IDs stored in Stripe's `product.metadata`). Adds tax and shipping as separate line items. Supports Stripe promotion codes.

`/api/webhooks/stripe` (POST)

Receives Stripe webhook events. On `checkout.session.completed`:

1. Retrieves line items from Stripe session

2. Separates products from tax/shipping line items
3. Maps Stripe products back to WooCommerce product IDs (via metadata)
4. Creates a WooCommerce order via REST API with billing/shipping details

`/api/validate-coupon` (POST)

Validates WooCommerce coupons: checks existence, usage limits, minimum/maximum amounts, and calculates discount (percent or fixed).

`/api/verify-code` (POST)

Product authentication system. Reads an Excel file (`peptiveverificationcode.xlsx`) to verify scratch codes. Once verified, marks the code as "used" in the spreadsheet. One-time use only.

`/api/calculate-shipping-tax` (POST)

Simple shipping calculator: free for UAE, AED 25 for GCC, AED 50 international. Tax is 5% calculated on the frontend.

`/api/proxy/wc-store/[...path]` (GET)

CORS proxy that forwards browser requests to WordPress Store API. Needed because browsers block cross-origin requests. Adds caching headers.

`/api/shipping-locations` (GET)

Fetches WooCommerce shipping zones and their locations to determine which countries/states you ship to.

`/api/customer-addresses` & `/api/customer-orders` (GET)

Fetch customer data from WooCommerce for the account page.

`/api/test-woocommerce` & `/api/test-wordpress` (GET)

Debug endpoints to test API connectivity. Not for production use.

Component Architecture

Layout Components (Always Visible)

Component	Purpose
`AnnouncementBar`	Top bar with promo message + country/language selectors
`Header`	Navigation, logo, cart icon, mobile menu drawer
`Footer`	Links, contact info, newsletter signup, payment icons
`CartSidebar`	Slide-out cart panel (right side)
`WelcomePopup`	Lead capture popup → submits to Google Forms

Product Components

Component	Purpose
`ProductGrid`	Responsive grid layout for product cards
`ProductCard`	Individual product card with image swap on hover, sale badges
`RelatedProducts`	Fetches and displays related products on detail page

UI Components

Component	Purpose
`Button`	Reusable button with variants (primary, secondary, outline, ghost)
`Input`	Form input with label and error state

Selectors

Component	Purpose
`CountrySelector`	Dropdown to select country (UAE, GCC, etc.)
`LanguageSelector`	Dropdown to switch between English and Arabic
`MobileLanguageToggle`	Compact EN/AR toggle for mobile header

Custom WordPress Plugin: `peptive-bundles`

Location in Repo

`wordpress-plugin/peptive-bundles/`

What It Does

This plugin adds **three features** to WooCommerce's admin panel and API:

1. Arabic Translation Fields (All Product Types)

Adds a new **"Arabic Translation"** tab in the WooCommerce product editor:

- Arabic Product Name (`_arabic_name`)
- Arabic Short Description (`_arabic_short_description`)
- Arabic Full Description (`_arabic_description`)
- Arabic Tags (`_arabic_tags`)

These fields are stored as post meta and exposed via both the REST API and Store API.

2. Monthly Bundle Pricing (Simple Products)

Adds a **"Monthly Pricing"** tab:

- 3-Month Regular Price (`_bundle_3_month_regular_price`)
- 3-Month Sale Price (`_bundle_3_month_sale_price`)
- 6-Month Regular Price (`_bundle_6_month_regular_price`)
- 6-Month Sale Price (`_bundle_6_month_sale_price`)

If left empty, the frontend auto-calculates (unit price × months).

3. Bundle Product Type

Registers a custom **"Product Bundle"** type in WooCommerce:

- Select component products and quantities
- Automatic inventory management (stock deducted from components)
- Bundle stock = lowest stock of any component
- Order display shows bundle contents
- Cart shows included products

Plugin File Structure

File	Purpose

File	Purpose
`peptive-bundles.php`	Main plugin file. Defines `WC_Product_Bundle` class, loads all includes.
`includes/class-bundle-product-type.php`	Registers "Product Bundle" type, adds admin tabs (Arabic, Pricing, Bundle Products), saves custom fields.
`includes/class-bundle-admin.php`	Enqueues admin scripts for product search.
`includes/class-bundle-api.php`	Extends both REST API and **Store API** with custom fields. Registers `peptive-bundles` namespace in Store API.
`includes/class-bundle-cart.php`	Shows bundle contents in WooCommerce cart.
`includes/class-bundle-order.php`	Shows bundle contents in order details and emails.
`includes/class-bundle-inventory.php`	Auto-deducts component stock on order, restores on cancel. Updates bundle stock status when components change.

How Plugin Data Reaches the Frontend

```
WordPress Admin → Saves post meta (_arabic_name,
_bundle_3_month_sale_price, etc.)
       │
       ▼
Store API Extension (class-bundle-api.php)
       │
       ├── Registers 'peptive-bundles' namespace via
woocommerce_store_api_register_endpoint_data
       └── Returns custom data in product.extensions['peptive-bundles']
              │
              ▼
Frontend (lib/woocommerce.ts → transformStoreProduct())
       │
       └── Extracts extensions data, maps to Product type
              │
              ▼
Product Detail Page shows Arabic content, bundle pricing options
```

`UPDATED-FUNCTIONS.php` — WordPress Theme Additions

This file contains PHP code that must be added to the active WordPress theme's `functions.php`. It provides:

1. CORS Configuration

Allows the Next.js frontend (on a different domain) to make API requests to WordPress. Supports:

- `http://localhost:3000` (development)
- `https://peptive.vercel.app` (production)

2. Store API Nonce Bypass

Disables the WooCommerce Store API nonce check, which is required for headless setups where the frontend is on a different domain.

3. Custom User Registration Endpoint

Registers `POST /wp/v2/users/register` — allows the frontend signup page to create new WordPress users with the "customer" role.

Internationalization (i18n)

The app supports **English** and **Arabic** (with RTL):

Source	Content
`lib/translations.ts`	All UI strings (buttons, labels, FAQs, etc.)
WordPress Plugin	Product names, descriptions, tags in Arabic
`LanguageContext.tsx`	Manages current language, `t()` function for translations

When Arabic is selected:

- `document.documentElement.dir` is set to `rtl`
- Product content switches to Arabic fields from the plugin
- All UI text switches via `t()` function

This document covers the complete technical structure of the Peptive Peptides codebase.